Auth Middleware & Protecting Routes

In Laravel, you can protect routes by using the `auth` middleware. As we saw in the last lesson, middleware is a way to filter HTTP requests entering your application.

Manually Checking User Authentication

You can still check if a user is authenticated manually by using the `Auth` facade. Let's say that you don't want users to be able to visit the `/jobs.create` route unless they are logged in. Right now, sure the button doesn't show, but they can still visit the route as a guest.

To protect the route manually, you can go into the `JobsController` `create` method and add the following:

Add the import:

```
use Illuminate\Support\Facades\Auth;
```

Then add the following to the `create` method. Also add `View | RedirectResponse` to the method signature.

```
 public function create(): View | RedirectResponse
{
    if (!Auth::check()) {
        return redirect()->route('login');
    }


    return view('jobs.create');
}
```

Now if you try to visit the `/jobs.create` route, you'll be redirected to the login page.

This is ok, but what if you want to protect multiple routes? You could copy and paste the code into each method, but that's not very DRY. Instead, you can use middleware. Delete the code we just added.

Using Middleware

Laravel comes with a built-in middleware for checking if a user is authenticated. It's called `auth`. You can use this middleware to protect routes.

To use the `auth` middleware, you can add it to the route definition in the `web.php` file.

Let's say that we want only logged in users to access the homepage, we would add the following to the `web.php` file:

```
Route::get('/', [HomeController::class, 'index'])->name('home')-
>middleware('auth');
```

Simple right? However, remember that we are using resource routes of jobs. We want to apply the auth middleware only to the create, store, edit, destroy and update routes. To do this, you can use the `only` method:

```
// Apply middleware to specific actions
Route::resource('jobs', JobController::class)->middleware('auth')->only(['create',
'edit', 'destroy']);
```

Now you will get an error like `jobs.show is not defined`. This is because we are using resource routes. To fix this, you can use the `except` method. Add this right under the code you just added:

```
// Define the rest of the resource routes without middleware
Route::resource('jobs', JobController::class)->except(['create', 'edit',
'destroy']);
```

Now you can visit the `/jobs.create` route and you will be redirected to the login page.

Next, we will look at the `guest` middleware as well as middleware groups.